COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 16 Processor architecture and assembly

Individual module / Expanded Volume 04

Four lessons with original foundations, expanded study, worked examples, laboratory checkpoints and quizzes.

Lessons 16.1–16.4 · 20 questions · One integration project

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Module 16: Processor architecture and assembly

Original prerequisites: Modules 11-15.

Integration goal: Machine-level investigation. Choose a bounded entity-data loop that produces a checked result. Explain its host assembly and compare it with the selected RISC-V teaching routine without treating the two targets as identical.

16.1 — Instruction sets, registers and calling conventions

Learning objective: Read a small machine-level routine without confusing ISA and implementation.

Original foundation

An instruction-set architecture defines visible operations, registers and relevant behavior. A microarchitecture implements that contract using particular pipelines, caches and execution units. Different implementations can run the same machine code while achieving different performance. A calling convention specifies how functions pass arguments, preserve registers and return values. Begin with a small load/store ISA such as the selected RISC-V subset, then compare your host's generated assembly. Do not assume an educational subset implements every extension or privileged feature. Assembly understanding helps explain costs and interoperability, but handwritten assembly is rarely the first optimization step.

Expanded study

Three contracts, not one

A compiler translates language rules into instructions; the instruction-set architecture defines what those instructions mean; the application binary interface defines how separately compiled routines cooperate. A correct arithmetic routine can still be unusable if it destroys a register the caller expects to survive. Conversely, a routine that obeys its calling convention may still compute the wrong sum. Keep a separate checklist for arithmetic, memory validity and call-boundary behavior. [R15, P01, P16]

Read the frame before reading the loop

The supplied RV32I routine accepts a pointer in a0 and a count in a1 and returns the low 32 bits of the sum in a0. It subtracts 16 from sp, saves ra at offset 12 and s0 at offset 8, then uses s0 as its accumulator. Those locations are relative to the adjusted stack pointer. On return it restores both registers and restores sp. Saving ra is redundant for this leaf function; it is included to make the frame exercise explicit, not as a performance recommendation. [P01, P16]

Prove the memory range

Let the original pointer be p and count be n. Before iteration k, the current pointer is p + 4k, the remaining count is n-k, and the accumulator contains the sum of the first k words modulo 2^32. The branch before the loop handles n=0 without a load. The caller must still provide readable aligned storage for all n words; the assembly routine cannot infer the true allocation length from a pointer. The C++ span expresses a view length, but its caller must also create a valid view.

Compare targets without pretending to emulate them

Run the host C++ example to check the arithmetic and inspect native/kernels.cpp in generated host assembly. Annotate native/asm/sum_u32_rv32.S separately. The C++ demonstration is not an instruction emulator, and assembling the RISC-V file is not the same as executing it on a RISC-V processor. Instruction spelling and register names will differ on x86-64 and Arm. Record the target triple and ABI whenever you save an assembly listing. [P02, P03]

Worked reasoning

For input words [7,11,0,5], the accumulator after each load/add is 7,18,18,23. The pointer advances by 16 bytes in total and the count reaches zero. With [4294967295,2], the result is 1 modulo 2^32. That machine-level wrapping contract is modeled with an unsigned C++ type; it is not permission to overflow a signed C++ int. Inspect all fifteen instruction lines in the supplied assembly, including the restore path, not just the five loop-body instructions.

Runnable example

File: `native/examples/lesson_16_1.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.1 --build-dir build
```

```
#include "lab_16_1.hpp"
#include <array>
#include <iostream>
int main(){std::array<std::uint32_t,4>a{7,11,0,5};std::cout<<"sum="<<course::sum_u32(a)<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
sum=23
```

Assembly annotation exercise

This is a separate RV32I teaching routine. The host demonstration does not emulate these instructions. Label the frame, caller preconditions, memory operations and restore path.

```
# Hand-authored RV32I teaching routine, ILP32 integer calling convention.
# Input a0: aligned pointer to n readable uint32 words; a1: n.
# Output a0: sum modulo 2^32. Caller provides valid memory and stack.
# ra save is deliberately redundant for this leaf; useful for frame annotation.
.text
.globl course_sum_u32
course_sum_u32:
    addi sp, sp, -16
    sw ra, 12(sp)
    sw s0, 8(sp)
    addi s0, zero, 0
    beq a1, zero, .Ldone
.Lloop:
    lw t0, 0(a0)
    add s0, s0, t0
    addi a0, a0, 4
    addi a1, a1, -1
    bne a1, zero, .Lloop
.Ldone:
    addi a0, s0, 0
    lw s0, 8(sp)
    lw ra, 12(sp)
    addi sp, sp, 16
    jalr zero, 0(ra)
```

Practical checkpoint

Annotate a ten-to-twenty-instruction routine with register meanings and memory accesses. Pass: predict its output for two inputs and identify caller/callee responsibilities from the chosen ABI reference.

Expanded contract: Native sum_u32(span<const uint32_t>) returns a modulo-2^32 sum; empty input returns zero and input is unchanged. The span must refer to valid live objects. No RISC-V execution is claimed by this helper.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_1.hpp`. Use `python3 check_lab.py 16.1 --student` after implementation.

Independent transfer

A caller keeps an important value in s0. Your sum routine returns 23 but leaves s0 equal to 23. Is the call correct?

Quiz — answer before opening the key

1. **How does an ISA differ from a microarchitecture?**
2. **What does a calling convention specify?**
3. **Why identify the exact ISA subset in a simulator?**
4. **What stack address contains the saved s0 after sp has been decremented by 16?**
5. **What result should the unsigned 32-bit reference produce for [4294967295,2], and why not use signed overflow?**

Assigned study

R15: Assembly Language and Procedures and Stacks; R35: selected base integer ISA.

Expansion source map: R15, R35, P01, P02, P03, P16. See the source directory for the exact documents and version notes.

16.2 — Pipelines, dependencies and branch prediction

Learning objective: Explain why instruction count alone does not predict latency.

Original foundation

A pipeline overlaps stages of multiple instructions. Dependencies can prevent later work from proceeding until earlier results are ready. Modern processors may issue or complete operations out of order while preserving required architectural behavior. Branch prediction speculates about control flow, and mispredictions can discard work. Throughput measures sustained work per time; latency measures delay for a particular result. A long dependency chain can have poor latency even when the processor can execute many independent operations per cycle. Do not infer performance from clock frequency alone. Instruction mix, memory stalls and available parallelism matter.

Expanded study

Build a dependency graph

Each addition to one accumulator consumes the previous accumulator value. Even if a processor can start multiple arithmetic instructions in one cycle, it cannot use a result that is not ready. Draw arrows between dependent operations before counting instructions. Loads may supply independent work, but their addresses, miss behavior and use dependencies can introduce other constraints. A branch adds control dependence, which prediction attempts to hide speculatively. [R15]

Use lower bounds honestly

In an invented machine model with four-cycle add latency, eight dependent additions have a 32-cycle dependency bound. With four independent two-add chains, the longest arithmetic chain is eight cycles. If the machine can issue one such addition per cycle, eight instructions also require at least eight issue cycles. These are lower bounds, not an exact prediction: starting the chains, loading operands, merging partial sums and recovering from branches still cost work.

Preserve semantics when exposing parallelism

The reference uses four unsigned 64-bit accumulators and combines them after the loop. Unsigned modular addition allows this regrouping. A floating-point sum needs a different contract because changing grouping can change rounding. Do not label a faster, numerically different result an improvement without deciding whether the difference is acceptable. The final tail also matters: seven elements include one four-element block and three remaining elements.

Measure the implementation that survived compilation

The native benchmark places kernels in a separate translation unit, produces a checked checksum, performs warmup and changes variant order. Those measures make the experiment interpretable, but assembly inspection is still required: an optimizing compiler might vectorize both variants or otherwise erase the difference you intended to test. A passing checksum establishes agreement on tested data, not a universal statement about every processor. Preserve raw trials instead of reporting only the fastest run. [P02, P03]

Worked reasoning

For [1,2,3,4,5,6,7], the first block gives accumulators (1,2,3,4). The tail adds 5,6,7 to the first accumulator, producing (19,2,3,4), whose sum is 28. This tail placement is deliberately simple; it does not mean the compiler issues the instructions in that order. Run benchmark in a Release build, save its CSV, and explain whether generated code still distinguishes the two source strategies.

Runnable example

File: `native/examples/lesson_16_2.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.2 --build-dir build
```

```
#include "lab_16_2.hpp"
#include <array>
#include <iostream>
int main()
{std::array<std::uint64_t,7>a{1,2,3,4,5,6,7};std::cout<<"sum="<<course::sum_split(a)<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
sum=28
```

Practical checkpoint

Compare dependent and independent arithmetic loops while preventing dead-code elimination. Pass: preserve observable results, inspect generated code and explain measured differences as evidence rather than universal CPU facts.

Expanded contract: Native sum_split(span<const uint64_t>) returns the modulo-2^64 sum without modifying input. All tail lengths are supported. Performance is an experiment, not a required speedup.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_2.hpp`. Use `python3 check_lab.py 16.2 --student` after implementation.

Independent transfer

Four accumulators are slower on a 12-element list. What should be concluded?

Quiz — answer before opening the key

1. **What is a data dependency?**
2. **Does higher clock frequency guarantee a faster program?**
3. **Why can extra independent accumulators help?**
4. **A model says add latency is four cycles and issue rate is two adds per cycle. What bounds apply to eight dependent adds?**
5. **Why is a checksum necessary but not sufficient for a benchmark?**

Assigned study

R15: Pipelining and Performance Measures.

Expansion source map: R15, R11, P02, P03. See the source directory for the exact documents and version notes.

16.3 — Caches, locality and memory hierarchy

Learning objective: Relate access patterns to cache lines and working sets.

Original foundation

Processors use a hierarchy of storage with different capacities and access costs. Caches transfer data in blocks called cache lines. Spatial locality reuses nearby data; temporal locality reuses recently accessed data. A working set is the data actively needed during a phase. Once it exceeds a cache level, behavior can change sharply. Cache associativity and replacement influence conflicts. A cache miss is not always a main-memory access, because another cache level may satisfy it. Prefetching can help predictable patterns but can also waste bandwidth. Measure several working-set sizes and avoid attributing every slow access to one guessed cache characteristic.

Expanded study

Translate addresses into model state

A direct-mapped cache model needs an explicit line size, number of lines, initial validity and replacement behavior. For byte address A, line size B and line count L, the memory block is floor(A/B), the cache index is block mod L, and the tag is floor(block/L). A hit requires both a valid line and a matching tag. Index equality alone is not a hit: different blocks compete for the same slot.

Separate locality from residency

Temporal locality reuses a recent value; spatial locality accesses nearby values that may share a fetched line. Both are properties of an access stream relative to a storage hierarchy. A large sequential pass can have good spatial locality even when little data is reused. Repeatedly revisiting a small set can have good temporal locality. Neither property guarantees a hit if competing accesses evict the data. [R15]

Avoid misleading stride comparisons

Reading every sixteenth element performs less useful work than reading every element. Comparing their elapsed times alone is not a fair locality experiment. The supplied locality benchmark instead visits all residue classes for a stride of sixteen so every element is visited exactly once. All three patterns use index arrays and check the same sum. Index-array traffic is included in the timed loop; this is a comparison of those complete access paths, not a pure measurement of DRAM latency.

Keep model and hardware reports separate

The teaching cache has two lines of sixteen bytes in the worked example. These are chosen model parameters, not information about the learner's machine. The host has additional cache levels, prefetchers, translation effects and shared-resource interference. A timing step can suggest a working-set boundary but cannot identify its cause uniquely. Record data bytes, index bytes, build settings and trial order. Do not use a tiny model to claim a precise cache-miss count for the real program. [R15, P02, P03]

Worked reasoning

With L=2 and B=16, address 0 maps to index 0/tag 0 and misses. Address 4 shares that line and hits. Address 32 maps to index 0/tag 1 and replaces it. Address 0 therefore misses again. Addresses 16 and 20 map to index 1/tag 0 and produce a miss then a hit. The complete sequence is M,H,M,M,M,H: two hits in six accesses. The model does not include dirty writeback, coherence or prefetching.

Runnable example

File: `native/examples/lesson_16_3.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.3 --build-dir build
```

```
#include "lab_16_3.hpp"
#include <iostream>
int main(){course::DirectCache c(2,16); for(std::uint64_t a:
{0,4,32,0,16,20})std::cout<<a<<':'<<(c.access(a)?"hit":"miss")<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
0:miss
4:hit
32:miss
0:miss
16:miss
20:hit
```

Practical checkpoint

Benchmark sequential, strided and randomized access over multiple array sizes. Pass: record assumed versus measured hardware information and explain spatial locality without claiming exact cache behavior from one timing curve.

Expanded contract: DirectCache(lines,line_bytes): lines in 1..65536, line_bytes in 1..1048576. access(uint64 address) returns a model hit and updates the selected line; reset invalidates all lines. Every modeled access fits in one line.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_3.hpp`. Use `python3 check_lab.py 16.3 --student` after implementation.

Independent transfer

Your timing curve slows at a particular array size. Does that prove the L2 size equals the array size?

Quiz — answer before opening the key

1. **What is spatial locality?**
2. **How many 4-byte values fit in a hypothetical 64-byte line?**
3. **Does every first-level cache miss access DRAM?**
4. **In the two-line, sixteen-byte model, why do addresses 0 and 32 conflict?**
5. **Why does the benchmark traverse every stride residue?**

Assigned study

R15: Caches and the Memory Hierarchy.

Expansion source map: R15, P02, P03. See the source directory for the exact documents and version notes.

16.4 — Virtual memory, SIMD and architectural limits

Learning objective: Recognize address translation and vector execution as separate mechanisms.

Original foundation

Virtual addresses belong to a process view; address translation maps them to physical storage subject to permissions. A translation lookaside buffer caches recent translations. Large working sets and access patterns can stress translation independently of ordinary data caches. Single-instruction multiple-data execution applies operations across several lanes. It benefits suitable regular work but needs attention to alignment, tails, masks and numerical semantics. Vector width is not a guaranteed speedup factor. Some workloads are limited by memory bandwidth or serial dependencies instead. Later co-design lessons will connect these observations to roofline and Amdahl-style limits.

Expanded study

Two different caches answer different questions

A data cache supplies recently used contents; a translation lookaside buffer supplies recent virtual-address translation information. A TLB miss is not automatically a missing data page. A page-table walk may find a present, permitted mapping, after which the data access can still hit or miss ordinary caches. A page fault can also be about permissions rather than disk I/O. Keep the address-translation path separate from the data path in a diagram. [R15, R16]

A lane diagram is not a speedup guarantee

A vector instruction can apply a suitable operation to several lanes, but the surrounding algorithm still loads operands, handles tails and stores results. Bandwidth, dependencies and instruction availability can limit the gain. The teaching add_blocks helper uses blocks of four scalar operations; it is not written with target-specific vector intrinsics. Only generated code or a documented target implementation establishes whether SIMD instructions are actually used. [P02, P03]

Alignment is a contract, not a cast

A span starting at the second uint32_t of a correctly allocated array is still properly aligned for uint32_t. It may no longer begin on a preferred vector boundary. That is a safe way to test an offset view. Casting an arbitrary byte address to a wider aligned type is not an equivalent experiment and can violate alignment or object-lifetime rules. Preserve typed storage and use legal accesses while studying compiler behavior.

Bound the last block and document aliasing

For n values and a block width W, complete blocks cover floor(n/W)*W values. The remainder must be processed without reading past the input or writing past the output. The helper permits disjoint output or exact whole-span aliasing with an input. Partial overlap is excluded: writing out[i] could overwrite a later input. A faster vector loop that silently relies on nonoverlap needs an explicit precondition, not an optimistic comment.

Worked reasoning

For virtual address 0x12345 and a chosen 4096-byte page size, the virtual page number is 0x12 and the page offset is 0x345. This arithmetic does not reveal a physical frame number. For five additions at width four, one complete block handles indices 0..3 and the tail handles index 4. The demo computes [11,22,33,44,55]. Sentinel elements around an offset output view must remain unchanged.

Runnable example

File: `native/examples/lesson_16_4.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 16.4 --build-dir build
```

```
#include "lab_16_4.hpp"
#include <array>
#include <iostream>
int main()
{std::array<std::uint32_t,5>a{1,2,3,4,5},b{10,20,30,40,50},o{};course::add_blocks(a,b,o);for(auto
v:o)std::cout<<v<<' ';std::cout<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
11 22 33 44 55
```

Practical checkpoint

Inspect compiler vectorization reports for a simple loop and compare aligned/misaligned or tail cases safely. Pass: test lengths 0, 1, lane_width-1 and lane_width+1 and explain translation versus caching.

Expanded contract: Native add_blocks(a,b,out) requires equal lengths and valid live spans; uint32_t addition wraps. Disjoint output or exact full alias is supported; partial overlap is outside the contract. Mismatched lengths throw before modification.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_16_4.hpp`. Use `python3 check_lab.py 16.4 --student` after implementation.

Independent transfer

The compiler refuses to vectorize an overlapping-buffer loop. Should the programmer simply assert nonaliasing?

Quiz — answer before opening the key

1. **What does a TLB cache?**
2. **Does four-lane SIMD guarantee a fourfold speedup?**
3. **Why test array lengths not divisible by vector width?**
4. **How many complete width-eight blocks and remaining elements occur for n=19?**
5. **Why can a TLB miss happen without disk access?**

Assigned study

R15: Virtual Memory and Parallel Processing; R11: bounds and arithmetic contracts.

Expansion source map: R15, R16, R11, P02, P03. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Trace a compiled C++ loop through a small assembly example and a cache model. Deliver an instruction/data-flow explanation and a controlled locality experiment.

Choose a bounded entity-data loop that produces a checked result. Explain its host assembly and compare it with the selected RISC-V teaching routine without treating the two targets as identical.

1. A register/memory trace for two inputs and the zero-length case; exact target and ABI.
2. A cache-model worksheet with stated line/entry assumptions and at least one conflict trace.
3. An equal-useful-work sequential/strided/random locality experiment at three working-set sizes.
4. A vectorization report and tail tests; explain whether observed assembly actually uses SIMD.

5. A report separating instruction semantics, analytical cost models and measured timing.

Live review: Change the line size in the model, then change the array length to one beyond a vector block. Predict the changed evidence before rerunning.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.